# Graphs and Search

**Topic 4 of 10**

**25 questions | skeleton and writing space only**

**Each question: the prompt, the function or class signature, and ruled lines to write your own solution.**

## 1. Friend Graph with Timeline (Friend / Unfriend Events)

**HARD | asked 2026-09-29 | graph, union-find**

**Examples: Base: N=3, events [(1,A,B,friend), (5,B,C,friend)] -> answer 5.**

```
def earliest_all_connected(n, events):
```

## 2. Infection Spread on a Grid (Cellular Automaton)

**HARD | asked 2026-06-14 |**

**It falls under algorithms and grid simulation, evaluating practical coding skill and the ability to handle progressively complex constraints including recovery delays, barriers, and early-termination fixed-point detection.**

```
def spread_steps(grid):
```

### 3. Edit distance

**HARD | asked n/a |**

**Determine the Edit Distance in a Word Ladder: Given two words (beginWord and endWord), and a dictionary's word list, find the minimum number of operations needed to change beginWord into endWord. You can change only one letter at a time, and each intermediate word must exist in the word list. If there is no possible transformation, return None (Python), -1 (Java & C++), null (Javascript).**

```
def edit_distance(a, b):
```

### 4. Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?

**HARD | asked n/a |**

**Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?**

```
def both_operations(s, t, alphabet=ALPHABET):
```

## 5. Template Variable Expansion with Follow-Ups

**MEDIUM | asked 2026-10-17 | string, parsing, graph, topological-sort, dfs, cycle-detection**

**Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion. Input: a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value. Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string.**

```
def expand_template(text, mapping):
```

## 6. Train / Flight Schedule Reachability

**MEDIUM | asked 2026-10-08 | graph, bfs, shortest-path**

**Given trains (or flights) as (from, to, departure, arrival), decide whether you can travel from a source to a destination, boarding a later leg only if it departs after you arrive. Graph reachability with time state, solved with BFS / DFS. Input: a schedule of legs, each with a departure station, an arrival station, a departure time, and an arrival time; plus a source and a destination.**

```
def earliest_arrivals(legs, source, strict=True
  ):
```

## 7. Course Scheduling and Dependency Variants

**MEDIUM | asked 2026-10-04 | graph, topological-sort, cycle-detection**

**Topological-ordering family. Variants: course ordering with validity checks and a nested-list extension; merging two arrays whose internal order defines precedence (return false on a conflict cycle); and a damaged-node dependency-path variant that minimizes broken nodes on a route. Course-ordering variant Work through the course-ordering problem, including validity checks, complexity analysis, and test cases. Extend the result to a nested-list format.**

```
def topological_order(vertices, edges):
```

## 8. Count Islands in a Grid and Track the Count as Land Cells Are Added

**MEDIUM | asked 2026-10-02 |**

**A map is a grid with m rows and n columns in which every cell is land (1) or water (0). An island is a maximal group of land cells connected horizontally or vertically. Cells that touch only at a corner are not connected. You are given the initial map and a sequence of additions, each of which turns one cell into land. Report the number of islands before any addition, and again after each addition.**

```
class Islands:

    def __init__(self):
```

```
    def find(self, x):
```

```
    def add(self, r, c):
```

```
    def union(self, a, b):
```

## 9. Number of Islands

**MEDIUM | asked 2026-09-30 | grid, dfs, bfs, union-find**

**Examples: [[1,1,0,0],[1,1,0,0],[0,0,1,1],[0,0,1,1]] -> 2. Streaming: [[0,0],[0,1],[1,1]] adds -> counts [1, 1, 1] (last addition merges the two land cells).**

```
def num_islands(grid):
```

## 10. Path Minimizing the Maximum Cell (Swim in Rising Water)

**MEDIUM | asked 2026-09-29 | graph, heap, algorithm**

**A bottleneck-path family: minimize the maximum value encountered on a grid path, then generalize the same reasoning to a weighted graph whose edges carry safety values and an admissibility threshold. Grid variant Input is a grid of heights and start/end cells; movement is through four-neighbors. Cost of a path is the maximum cell value along it. Return the minimum possible path cost.**

```
def min_max_path(g, start, end):
```

## 11. Shortest Path Nodes and Escaping a Pursuer

**MEDIUM | asked 2026-09-29 | graph, bfs, shortest-path**

**Unweighted undirected graph: shortest path from Alice's node to a target (BFS). Input: an undirected, unweighted graph; Alice starts at one node and must reach a target node. Return the shortest path (length) from Alice to the target. Alice picks one shortest path and starts moving; Bob starts moving at the same time. Decide whether Alice can reach the target safely without being caught by Bob.**

```
def shortest_length(adj, a, t):
```

## 12. Rotten Oranges / Multi-Source BFS (taxis)

**MEDIUM | asked 2026-09-08 | grid, bfs**

**Examples: Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-oranges variant): return number of minutes until all reachable 1s become 2 (4). Taxi variant: for each empty cell, distance to nearest taxi.**

```
def rotting_minutes(grid):
```

## 13. 2D Points Connected Components by Radius

**MEDIUM | asked 2026-08-04 | graph, dfs, bfs, union-find, clustering**

**Given 2D points and radius r, connect two points when their distance is at most r; connectivity is transitive, and the output is the final number of clusters. Input: a list of 2D points and a radius r. Two points are connected if their Euclidean distance is <= r. Connectivity is transitive: if A connects to B and B connects to C, all three belong to the same cluster. Return the number of final clusters.**

```
def count_clusters(points, r):
```

## 14. Minimum Direction Violations in a Directed Graph

**MEDIUM | asked 2026-07-19 | graph, bfs, dijkstra**

**Examples: n = 5 edges = [[0, 1], [2, 1], [2, 3], [4, 3]] start = 0 end = 4 path: 0 -> 1 -> 2 -> 3 -> 4 cost: 0 + 1 + 0 + 1 = 2 output: 2**

```
def min_violations(n, edges, start, end):
```

## 15. Count Clusters of 2D Points Within a Radius

**MEDIUM | asked 2026-06-18 |**

**It assesses the ability to model a geometric problem as a graph and apply DFS, BFS, or Union-Find, a common evaluation of algorithmic thinking in coding interviews. It also probes awareness of efficiency trade-offs when scaling to large point sets.**

```
def count_clusters(points, r):
```

## 16. Find shortest relationship path using BFS

**MEDIUM | asked 2026-02-12 |**

**Overview: This question evaluates proficiency in graph representations and traversal algorithms--specifically BFS and adjacency-structure construction--for computing unweighted shortest paths and reasoning about reachability.**

```
def shortest_path(adj, s, t):
```

## 17. Validate parent array forms a tree

**MEDIUM | asked 2026-02-11 |**

**## Problem You are given an integer array parent of length n describing a directed parent pointer for each node i (nodes are labeled 0..n-1). - parent[i] is the parent of node i. - Exactly one node should be the root, indicated by parent[root] = -1. Determine whether these n nodes form a **valid rooted tree**. A valid rooted tree must satisfy all of the following: 1. **Exactly one root** (exactly one index i with parent[i] = -1). 2.**

```
def validate_parent_array(parent):
```

## 18. Determine whether two nodes are related

**MEDIUM | asked 2026-01-22 |**

**Overview: This question evaluates understanding of graph representations, reachability and connectivity concepts for ancestry relationships, including handling disconnected components and potential cycles.**

```
class Relations:

    def __init__(self, n, edges=()):

    def find(self, x):

    def union(self, a, b):

    def related(self, a, b):
```

## 19. Solve matrix groups and recipe inventory

**MEDIUM | asked 2026-01-22 |**

**Overview: This pair of problems evaluates block-wise grid processing and aggregation for minimum-value selection alongside dependency-resolution and resource-allocation reasoning for recipe inventory planning, testing competencies in array manipulation, tie-breaking and constraint handling, graph-based dependency management and cycle detection, and algorithmic complexity analysis; they are commonly asked to gauge an applicant's ability to implement correct, efficient solutions under input-size and edge-case constraints.**

```
def block_mins(matrix, k):
```

## 20. Compute distance to nearest taxi in grid

**MEDIUM | asked 2026-01-10 |**

**Overview: This question evaluates understanding of graph traversal and shortest-path reasoning on grids, algorithmic problem-solving for computing distances from many targets, and the ability to manage time and space complexity on large inputs.**

```
def taxi_distances(grid):
```

## 21. Validate course catalog dependencies

**MEDIUM | asked 2025-08-01 |**

**Design a function to validate an e-learning course catalog. You are given: ( 1) a set of course IDs, and ( 2) a list of prerequisite pairs (u, v) meaning course v requires course u. Determine if the catalog is valid: no prerequisite references a non-existent course, and there are no cycles in the dependency graph. If valid, return true and a possible ordering of courses; otherwise return false and report the set of missing course references and at least one cycle you detect.**

```
def validate_catalog(courses, prereqs):
```

## 22. Solve Grid and Counting Problems

**MEDIUM | asked 2025-01-25 |**

**A software engineering intern phone screen included two coding tasks: 1. **Count islands in a grid** Given an m x n grid of characters where '1' represents land and '0' represents water, return the number of islands. An island is formed by connecting adjacent land cells horizontally or vertically. You may assume all four edges of the grid are surrounded by water. 2.**

```
def num_islands(grid):
```

## 23. Shortest Directed Cycle Through a Node

**MEDIUM | asked n/a | graph, bfs**

**Examples: Edges [(0,1),(1,2),(2,0)], v = 0 -> 3. Edges [(0,1),(1,2)], v = 0 -> -1.**

```
def shortest_cycle_through(n, edges, v):
```

## 24. Course Schedule

**MEDIUM | asked n/a |**

**You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number.**

```
def can_finish(n, prereqs):
```

## 25. Coin Change

**MEDIUM | asked n/a |**

**You are given an integer array coins representing different coin denominations and an integer amount representing a total amount of money. Write a function coinChange that returns the fewest number of coins needed to make up that amount. If that amount cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin.**

```
def coin_change(coins, amount):
```